VENUS REMEDIES · INTERNAL KNOWLEDGE CHATBOT

ARCHITECTURE

App flow, folder structure, and technology choices

Project: Internal Knowledge Chatbot — Venus Remedies

1. App Flow & Architecture

Ingestion pipeline (separate, admin-triggered flow): Document upload → text extraction → chunking → embedding generation → store chunks + embeddings + source metadata in vector DB.

2. Folder & File Structure

```
knowledge-chatbot/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── ChatWindow.jsx
│   │   │   ├── MessageBubble.jsx
│   │   │   ├── SourceCitation.jsx
│   │   │   └── LoginForm.jsx
│   │   ├── pages/
```

```
│   │   │   ├── Login.jsx
│   │   │   └── Chat.jsx
│   │   ├── lib/
│   │   │   └── api.js          # thin wrapper around backend calls
│   │   ├── App.jsx
│   │   └── main.jsx
│   └── package.json
│
├── backend/
│   ├── src/
│   │   ├── routes/
│   │   │   ├── auth.js
│   │   │   ├── chat.js
│   │   │   └── ingest.js       # admin-only document upload endpoint
│   │   ├── services/
│   │   │   ├── retrieval.js    # vector search logic
│   │   │   ├── llm.js          # LLM API call wrapper
│   │   │   └── chunking.js     # document → chunk logic
│   │   ├── middleware/
│   │   │   └── requireAuth.js
│   │   └── server.js
│   └── package.json
│
├── docs/                        # this planning doc set lives here
│   ├── Project-Requirements.md
│   ├── Architecture.md
│   ├── Rules.md
│   ├── Phases.md
│   ├── Design.md
│   └── Memory.md
│
└── README.md
```

3. Tech Stack (recommended, adjust if company has existing standards)

LAYER	CHOICE	WHY
Frontend	React (Vite)	Fast to build, team likely already familiar, matches Phase 1/2 scope well
Backend	Node.js + Express	Simple REST API, easy to reason about, good LLM SDK support
Vector DB	Supabase (Postgres + pgvector)	One system for both regular data (users, sessions) and vector search — avoids running two separate databases for a v1
LLM	Claude API (Anthropic)	Company context suggests Claude usage already; strong grounded-answer behavior
Auth (v1)	Simple email+password or magic link	Company SSO integration is a v2 concern, not needed to prove the concept
Hosting	Vercel/Netlify (frontend) + Render/Railway (backend)	Fast to deploy, low ops overhead for a v1 demo

Note: Keep the vector DB and the LLM provider decoupled from the rest of the app logic — if the company later wants to switch LLM providers or self-host a model for compliance reasons, that swap should only touch `services/llm.js`, nothing else.